

NVIDIA Jetson Orin Nano 8GB

Local GPU Inference Node (LLM + Speech-to-Text)

Installation guide: boot to NVMe, minimise the OS, build llama.cpp and whisper.cpp natively for CUDA, and serve a local LLM + STT over the network for Home Assistant and a separate mini-PC orchestrator.

Key decisions captured in this build

- Scope: the Jetson is an INFERENCE NODE only. Host only GPU models that run on the llama.cpp/ggml stack. Everything else lives on a separate mini-PC.
- Install: JetPack 7.2 'Jetson ISO' on a USB stick installs Jetson Linux directly onto the NVMe (no SD clone, no x86 Linux host).
- Runtime: NATIVE builds (no Docker) - llama.cpp (LLM) and whisper.cpp (STT), both CUDA sm_87, served as systemd services.
- Models: Qwen3-8B Q4_K_M (text, :8080) and Whisper large-v3-turbo Q5_0 (STT, auto NL/EN, :8081).
- Minimal + efficient: headless multi-user target, non-essential services disabled, MAXN_SUPER persistent with DVFS left on (no 24/7 jetson_clocks).
- Off-Jetson (mini-PC): Piper TTS, the OpenClaw agent harness, browser automation and approval gates.

Version 2.0 - generated 2026-06-23. Pin versions in versions.env; treat this as living documentation.

1. Overview & Architecture

This guide provisions an always-on GPU inference node on a Jetson Orin Nano 8GB with a 500GB NVMe SSD. The Jetson does ONE job well: serve a local LLM and speech-to-text on the GPU behind simple HTTP endpoints. A separate mini-PC runs the agent harness (OpenClaw), text-to-speech and all orchestration, and calls the Jetson over the LAN. Keeping the Jetson single-purpose is what lets the 8GB box stay lean and reliable.

What runs WHERE

Component	Host	Endpoint
LLM (Qwen3-8B, llama.cpp)	Jetson GPU	:8080 /v1
STT (Whisper large-v3-turbo, whisper.cpp)	Jetson GPU	:8081 /inference
TTS (Piper)	mini-PC / HA	Wyoming
Agent harness (OpenClaw), browser, approvals	mini-PC	-
Home Assistant (Assist pipeline)	HA host	-

Why only LLM + STT on the Jetson

- Both genuinely benefit from the GPU and both run on the same ggml/llama.cpp ecosystem - one toolchain, one build recipe.
- Piper (TTS) is a light CPU/ONNX task with no real GPU benefit, so it stays off the Jetson to protect the 8GB budget.
- With both models loaded the box already sits at ~6.9/7.3 GB RAM - there is no room for a third large model.

Architecture (data flow)

```

Home Assistant (Assist)      mini-PC (orchestrator)
| STT audio                  | chat / tools / approvals
v                             v (OpenClaw, Piper TTS, browser)
Jetson Orin Nano 8GB (LAN, no inbound router ports)
- llama-server :8080 -> OpenAI-compatible /v1 (Qwen3-8B)
- whisper-server :8081 -> /inference (STT, auto NL/EN)
systemd-managed, GPU-accelerated, auto-start on boot

```

Design principles

- Single purpose: the Jetson only does GPU inference; orchestration and side-effects live on the mini-PC.
- Native over containers: build llama.cpp/whisper.cpp directly against the host CUDA - no Docker layer to debug.
- Lean OS: headless, non-essential services off, energy-aware power management (DVFS on).
- Version pinning: reproducible builds via versions.env; avoid 'latest' for anything load-bearing.

Why native, not Docker (JetPack 7 reality)

On JP7 (L4T r39.2) there is no current prebuilt dusty-nv llama.cpp image (tags stop at r36/JP6). The JP6 CUDA images either fail GPU init or segfault on JP7. Building natively against the host CUDA 13.2 is the reliable path and also reclaims the ~130MB Docker/containerd footprint. A Docker fallback note is in the appendix.

2. Hardware Checklist & Memory Budget

Hardware

- Jetson Orin Nano 8GB Developer Kit (Super-capable; JetPack 7.2 / L4T r39.2, board p3767-0003).
- NVMe M.2 SSD 500GB (PCIe Gen3) installed in the M.2 Key-M slot - becomes the root filesystem.
- USB stick (≥ 16 GB) for the one-time JetPack installer (throwaway boot media).
- DisplayPort monitor + USB keyboard for the one-time install (no video over USB-C); headless via SSH afterwards.
- Active cooling (fan) - required for sustained inference.
- Included 19V power supply (barrel jack); MAXN SUPER needs the full supply - don't rely on weak USB-C PD.
- Wi-Fi or Ethernet. (This build was provisioned over Wi-Fi; a DHCP reservation for the Jetson IP is recommended.)

Measured memory budget (8GB unified - real numbers from this build)

Component	Approx. RAM
OS (headless, minimised)	~0.7 GB
LLM: Qwen3-8B Q4_K_M + KV (c=8192, q8_0)	~5.2 GB
STT: Whisper large-v3-turbo Q5_0	~0.55 GB
Both services loaded (measured)	6.9 / 7.3 GB used
Free with both loaded	~0.4 GB + 16 GB swap

[!] 8GB is the hard constraint

Qwen3-8B Q4 + Whisper turbo + the OS already use ~6.9GB. Keep a 16GB swap file on the NVMe as an OOM safety net, do NOT add a third large model, and keep TTS/agent/browser workloads on the mini-PC. If you need more LLM headroom, lower `--ctx-size` or use a smaller quant.

3. Phase 1 - Install JetPack to the NVMe (USB-ISO)

Since JetPack 7.2 (L4T r39.2, mid-2026), NVIDIA ships a 'Jetson ISO' installer that you write to a USB stick. You boot the Jetson once from that USB stick and the installer updates the QSPI firmware and installs Jetson Linux DIRECTLY onto the NVMe SSD - no SD card, no rootfs clone, no x86 Linux host. The USB stick is throwaway boot media (like a DVD); remove it afterwards.

On Windows: write the ISO to a USB stick

1) Download the Jetson ISO from NVIDIA (developer.nvidia.com -> JetPack downloads -> Jetson ISO r39.2). 2) Write it to a >=16GB USB stick with balenaEtcher (Flash from file -> Select target = the USB stick -> Flash). Verify the download (size / SHA256). The firmware update is baked into the ISO - no internet needed for it.

[!] No video over USB-C

The Orin Nano Developer Kit outputs display only over its DisplayPort connector (no HDMI port; USB-C does NOT carry video). The UEFI firmware prompt cannot be driven over USB-C, so you need a DisplayPort monitor + USB keyboard (or a USB-to-TTL serial cable) for the one-time install. Afterwards the device runs fully headless via SSH.

On the Jetson: boot the USB and install

- Attach a DisplayPort monitor + USB keyboard, network, the NVMe, and the USB stick, then connect the included 19V power supply.
- Press Esc at the NVIDIA splash -> Boot Manager -> select the USB stick.
- Press Y within 30 seconds to accept the QSPI firmware capsule update (the most-missed step; the install fails later if skipped). It runs in two passes with reboots.
- At the GRUB menu choose 'Install Jetson ISO r39.2', select the NVMe SSD as the target (not the USB, not a microSD), and confirm (this erases the NVMe).
- Reboot when prompted and remove the USB stick - the system now runs entirely from the NVMe.

On first boot from the NVMe, complete the Ubuntu setup (EULA, language, network, create your user + hostname). SSH (openssh-server) is enabled by default on L4T, so the device is reachable over the network immediately after this step.

4. Phase 2 - System Minimisation & Power

Run on the device after booting from NVMe. Goal: a lean, headless server that keeps the GPU at a high ceiling without wasting energy. See scripts/01-system-config.sh and scripts/02-swap.sh.

Headless + remove the desktop

```
sudo systemctl set-default multi-user.target      # no GUI on boot
sudo apt purge -y ubuntu-desktop gnome-shell gdm3 yaru-theme-*
sudo apt autoremove --purge -y && sudo apt clean  # frees several GB
```

Power mode: MAXN SUPER, made persistent

nvpmodel sets a persistent power CEILING; jetson_clocks merely PINS max clocks (disables DVFS) and is deliberately non-persistent. We want the high ceiling but keep DVFS on so the GPU idles down and saves energy.

```
sudo nvpmodel -m 2          # MAXN SUPER (mode 2 on p3767-0003 super conf)
# Persistence gotcha: symlinking /etc/nvpmodel.conf is reset on boot.
# Make MAXN_SUPER the board default by editing the board conf itself:
#   /etc/nvpmodel/nvpmodel_p3767_0003.conf -> set 'DEFAULT 2'
# (back up the original as .orig first). Do NOT enable a jetson_clocks
# service - leave DVFS (governor schedutil) on to save energy.
```

Disable non-essential services

Audit with 'systemctl list-unit-files --state=enabled' and 'systemd-analyze blame'. On a headless inference node you can safely disable, e.g.: bluetooth, ModemManager, avahi-daemon, cups/lpd, kerneloops, apport, gnome-remote-desktop, switcheroo-control, power-profiles-daemon, accounts-daemon, isc-dhcp-server, dnsmasq, openvpn, sssd, ubuntu-advantage, anacron, fwupd, nvargus-daemon (cameras), udisks2, and the host audio stack (pipewire/wireplumber). Mask snapd and disable cloud-init if unused.

Swap on NVMe (OOM safety net)

```
sudo ./scripts/02-swap.sh 16      # 16GB /swapfile on the NVMe
sudo sysctl vm.swappiness=10      # persist in /etc/sysctl.d
```

Note - 250MB is not reachable, and that's fine

Even a bare L4T install has an irreducible ~210MB of OS processes plus a kernel/GPU memory carveout. You cannot swap to Alpine/buildroot and keep the GPU - the NVIDIA driver only ships for L4T (Ubuntu 24.04). The realistic floor is what matters: ~0.7GB OS, leaving ~6.5GB for models.

5. Phase 3 - Native CUDA Build Toolchain

JetPack 7.2 ships the GPU driver (CUDA 13.2 runtime) but NOT a full CUDA toolkit. Install a MINIMAL build toolchain so we can compile llama.cpp and whisper.cpp against the GPU. Match the toolkit to the driver (CUDA 13.2).

```
sudo apt update
sudo apt install -y cuda-minimal-build-13-2 cuda-nvrtc-dev-13-2 \
    libcublas-dev-13-2 cmake g++ libcurl4-openssl-dev ccache git
export PATH=/usr/local/cuda-13.2/bin:$PATH
export CUDACXX=/usr/local/cuda-13.2/bin/nvcc
nvcc --version      # expect release 13.2
```

[!] cuBLAS is required

Without libcublas-dev-13-2 the CMake configure fails with 'CUDA::cublas target not found'. The minimal-build metapackage does NOT pull it in - install it explicitly.

6. Phase 4 - LLM: build & serve llama.cpp

Build llama.cpp natively for the Orin's compute capability 8.7 (sm_87), then serve Qwen3-8B (Q4_K_M) over an OpenAI-compatible API on :8080.

Build (CUDA sm_87)

```
cd /opt && git clone --depth 1 https://github.com/ggml-org/llama.cpp.git
cd llama.cpp
cmake -B build -DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=87 \
  -DLLAMA_CURL=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc) --target llama-server llama-cli
# The CUDA kernel/flash-attn compile is the long part (~25-35 min).
```

Model

- Qwen3-8B-Q4_K_M.gguf (strong NL/EN; ~4.8GB). Store under /opt/models.
- Download from a trusted GGUF source; verify it loads (llama-cli) and pin the filename in versions.env.

Serve as a systemd service

```
# /etc/systemd/system/llama-server.service (ExecStart, one line):
ExecStart=/opt/llama.cpp/build/bin/llama-server \
  -m /opt/models/Qwen3-8B-Q4_K_M.gguf -ngl 99 -c 8192 -fa on \
  --cache-type-k q8_0 --cache-type-v q8_0 \
  --host 0.0.0.0 --port 8080 --alias qwen3-8b
Environment=LD_LIBRARY_PATH=/usr/local/cuda-13.2/lib64

sudo systemctl enable --now llama-server
curl http://127.0.0.1:8080/v1/models      # health check
```

Expected performance

On the Orin Nano Super, Qwen3-8B Q4 generates ~7.5-9 tok/s. Token generation is memory-bandwidth bound (~102 GB/s / ~4.8GB model ~21 tok/s theoretical ceiling, ~40-60% realised). This is normal and comfortably faster than reading speed for an assistant.

7. Phase 5 - Speech-to-Text: build & serve whisper.cpp

Same ggml ecosystem, same CUDA recipe. whisper.cpp gives GPU-accelerated STT with automatic language detection (NL/EN) on :8081.

Build (CUDA sm_87)

```
cd /opt && git clone --depth 1 https://github.com/ggml-org/whisper.cpp.git
cd whisper.cpp
cmake -B build -DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=87 \
  -DWHISPER_BUILD_SERVER=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc) --target whisper-server whisper-cli
```

Model + serve

```
cd /opt/models && curl -L -o ggml-large-v3-turbo-q5_0.bin \
  https://huggingface.co/ggerganov/whisper.cpp/resolve/main/ggml-large-v3-turbo-q5_0.bin

# /etc/systemd/system/whisper-server.service (ExecStart):
ExecStart=/opt/whisper.cpp/build/bin/whisper-server \
  -m /opt/models/ggml-large-v3-turbo-q5_0.bin -l auto -t 4 \
  --host 0.0.0.0 --port 8081
Environment=LD_LIBRARY_PATH=/usr/local/cuda-13.2/lib64

sudo systemctl enable --now whisper-server
# test: curl -F file=@sample.wav -F language=auto \
#   -F response_format=json http://127.0.0.1:8081/inference
```

- large-v3-turbo-q5_0 (~0.55GB) balances accuracy and footprint; it auto-detects NL vs EN.
- The GPU backend (CUDA0) is selected automatically when built with -DGGML_CUDA=ON.

8. Phase 6 - Home Assistant Integration

Home Assistant orchestrates voice. The Jetson provides the LLM and STT; TTS (Piper) and the agent live elsewhere.

Wire it up

- LLM: install 'Extended OpenAI Conversation' (HACS) and set base_url to `http://<jetson-ip>:8080/v1` - this becomes the Assist conversation agent.
- STT: bridge the `whisper.cpp` :8081 HTTP endpoint into HA via a Wyoming/whisper bridge (or run a Wyoming-Whisper satellite that posts to it).
- TTS: run Piper on the mini-PC or as an HA add-on (Wyoming) - NOT on the Jetson.
- Build an Assist pipeline: STT -> conversation agent (Jetson LLM) -> Piper TTS.

Microphone & speakers

- Sonos is OUTPUT only: use it as a `media_player/announce` target for TTS. Its built-in mic (and Google Nest) cannot feed HA Assist - no usable API, and Google-Assistant-on-Sonos was discontinued in 2024.
- For voice INPUT you need a Wyoming satellite: Home Assistant Voice PE (recommended) or an ESP32 'Atom Echo'. The satellite streams mic audio to the pipeline.

Approval boundary

Voice/HA must not trigger high-impact agent actions (orders/payments) directly. Those go through the OpenClaw approval gate on the mini-PC. Keep HA and the Jetson on the LAN/VPN only - do not expose :8080/:8081 to the internet.

9. The Orchestration Layer (separate mini-PC)

The agent harness does NOT run on the Jetson. A separate mini-PC runs OpenClaw (skills, browser, messaging), Piper TTS, and personalization, and calls the Jetson's :8080 LLM as one of its model backends. This split keeps side-effects and credentials off the inference node. Full OpenClaw setup and the security/approval model are documented in the repo (config/openclaw, SECURITY.md); the essentials:

- OpenClaw runs sandboxed (non-main sessions, sandbox egress network: none); the browser uses an isolated, credential-free profile.
- High-impact actions (payments, orders, messaging third parties, deleting data) require explicit human approval and the agent must never work around a denied/pending approval.
- Least privilege: the agent holds NO payment/order credentials, so a blocked action cannot complete without you - structural, not just a prompt rule.
- Personalization is done with RAG + a memory layer on the mini-PC (retrieve personal context at query time), NOT by finetuning on the Jetson.

Why not finetune on the Jetson

Finetuning an 8B model (even QLoRA) realistically needs ~12-16GB and heavy training throughput the Orin Nano doesn't have. For 'it knows me', use RAG + memory on the mini-PC. If you truly need weight adaptation, train a LoRA off-box (cloud NVIDIA GPU) and load it with llama-server --lora; the Jetson stays inference-only.

10. Verification & Benchmarking

```
lsblk                                # root on nvme0n1p1
nvpmodel -q                         # MAXN SUPER, persists after reboot
systemctl is-active llama-server whisper-server  # both 'active'
systemctl is-enabled llama-server whisper-server  # both 'enabled'
free -h                             # expect ~6.9/7.3GB with both loaded
tegrastats --interval 1000          # GR3D_FREQ shows GPU use under load

# LLM round-trip (Dutch prompt):
curl -s http://127.0.0.1:8080/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -d '{"model": "qwen3-8b", "messages": [{"role": "user", \
    "content": "Hoofdstad van Nederland?"}], "max_tokens": 20}'

# STT round-trip:
curl -F file=@/opt/whisper.cpp/samples/jfk.wav -F language=auto \
  http://127.0.0.1:8081/inference
```

11. Maintenance & Updates

- Update the LLM model: download a new GGUF to /opt/models, update versions.env + the service ExecStart, 'sudo systemctl restart llama-server'.
- Update the engines: 'git pull' in /opt/llama.cpp or /opt/whisper.cpp, rebuild (same cmake recipe), restart the service. Pin the commit in versions.env.
- Keep the OS current: 'sudo apt update && sudo apt full-upgrade' (includes firmware capsules).
- After a CUDA toolkit bump, rebuild both engines and update LD_LIBRARY_PATH in the unit files.
- Commit every change to this deployment repo so the running state stays captured.

12. Troubleshooting

Symptom	Fix
NVMe won't boot after install	Re-run the USB installer; ensure you pressed Y for the firmware capsule; check UEFI boot order
CMake: 'CUDA::cublas not found'	Install libcublas-dev-13-2 (minimal-build doesn't pull it in)
nvcc not found	Add /usr/local/cuda-13.2/bin to PATH (and a profile.d entry)
MAXN_SUPER resets on boot	Edit the board conf nvpmode1_p3767_0003.conf (DEFAULT 2), not the /etc symlink
OOM / killed processes	Lower --ctx-size, keep 16GB swap, don't add a 3rd model
Port already in use on restart	Free it via 'sudo fuser -k 8080/tcp'; avoid 'pkill -f llama-server' (matches your own ss) (killall llama-server)
Docker CUDA image fails on JP7	Use the native build; if you must use Docker see the appendix (nvgpu LD_LIBRARY_PATH)

Appendix A - Version Pin Table

Mirror of versions.env. Update deliberately; verify on-device before bumping.

Item	Pinned value
JetPack	7.2
L4T	r39.2
CUDA toolkit / driver	13.2 (sm_87 / compute 8.7)
LLM engine	llama.cpp (native, pin commit)
LLM model	Qwen3-8B-Q4_K_M.gguf
STT engine	whisper.cpp (native, pin commit)
STT model	ggml-large-v3-turbo-q5_0.bin
TTS (off-Jetson)	Piper (mini-PC / HA)
Agent harness (off-Jetson)	OpenClaw (mini-PC)

Appendix B - Docker fallback on JetPack 7 (if ever needed)

Native is the recommended path. If you must run a JP6 (r36.4) CUDA container on the JP7 host, the GPU only initialises when you force it to use the nvgpu libcuda; the default/openrm variant reports 'no CUDA-capable device'.

```
# inside / for the container, point the loader at the nvgpu libs:
LD_LIBRARY_PATH=/opt/nvidia/l4t-gpu-libs/nvgpu <cuda-program>
# Note: older engine builds may not know newer architectures
# (e.g. r36.4.0 llama.cpp errors 'unknown model architecture: qwen3').
```